



La grande école du numérique pour tous

# KickOff Lancement Chaîne unique





# Chaîne unique

Une seule machine industrielle, des délais contractuels stricts, et des pénalités financières à minimiser de façon scientifique.

## Un goulet d'étranglement critique

Dans cet atelier de reprographie, une seule chaîne réalise l'impression, le façonnage et la reliure. Rien ne tourne en parallèle et aucune interruption n'est autorisée.

**Problème actuel :** Le chef d'atelier trie naïvement par date d'échéance. Chaque fin de mois, l'entreprise paie de lourdes pénalités sans savoir si elles étaient évitables.

**Votre mission :** comparer des règles d'ordonnancement, quantifier l'écart à l'optimum et livrer la règle à appliquer.

# Anatomie d'une commande | Règles du Jeu & Contraintes

## Durée de fabrication (p)

Temps continu nécessaire pour traiter entièrement la commande sur la chaîne unique. Aucun temps mort ni découpage possible.

## Échéance limite (d)

Date de livraison contractuelle ferme négociée avec le client. La commande doit impérativement s'achever à cet instant ou avant.

## Pénalité financière (w)

Montant forfaitaire dû en totalité dès que l'échéance est dépassée. Un retard d'une minute coûte autant qu'un retard d'une semaine.

**Principe du Tout ou Rien :** La pénalité **w** est entièrement due si  $\text{fin} > \text{d}$ . Il n'existe pas de pénalité proportionnelle au temps de retard.

**Mesures algorithmiques pures :** On compte des pénalités et des commandes sauvées, jamais des temps en secondes. Les benchmarks matériels sont exclus.

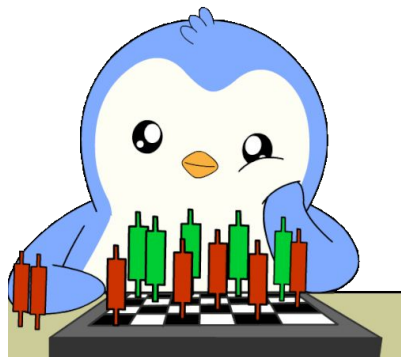
**Interdiction absolue des solveurs :** Tout solveur externe (SCIP, OptaPlanner, HiGHS, solveurs LP) est **banni**. Seules vos structures natives sont autorisées.

**Règle de départage imposée :** En cas d'ex æquo lors du retrait d'une tâche, retirez celle de plus longue durée **p**, puis celle apparue le plus bas dans le fichier source.



# Les Quatre Stratégies Métier

| Règle | Critère de retrait en cas de retard                   | Objectif visé                                    | Statut Mathématique       |
|-------|-------------------------------------------------------|--------------------------------------------------|---------------------------|
| R1    | Aucun (elle ne retire rien)                           | Ordre par échéance brute (pratique actuelle)     | Témoin naïf               |
| R2    | La commande de plus longue durée ( <b>p</b> max)      | Maximiser le nombre de commandes à l'heure       | Optimal prouvé            |
| R3    | La commande de plus faible pénalité ( <b>w</b> min)   | Minimiser le coût financier des pénalités        | Heuristique sans garantie |
| R3'   | Le plus faible ratio pénalité/durée ( <b>w/p</b> min) | Optimiser le compromis valeur financière / temps | Heuristique avancée       |



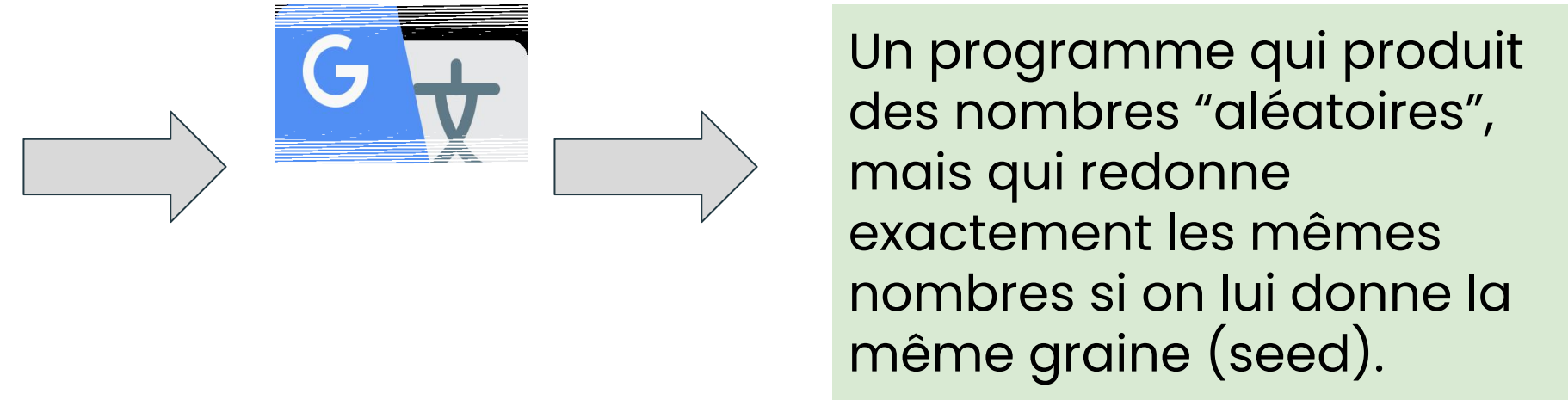
# Générateur

## Étape 1 : Le Générateur

- Générateur 32 bits reproductible (xorshift32 ou congruentiel) piloté par une graine.
- **Technique de plantation** : fabriquer un lot garanti faisable, consigner sa taille dans un témoin séparé, puis le noyer dans du bruit.
- Le solveur ne doit jamais avoir accès au fichier témoin.



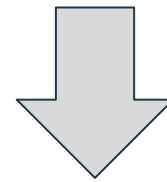
- **Xorshift** → utilise des opérations très simples sur les bits (**XOR**, décalages).
- **Congruentiel** → utilise une formule mathématique :  
$$\text{nouveau} = (a \times \text{ancien} + c) \% m$$



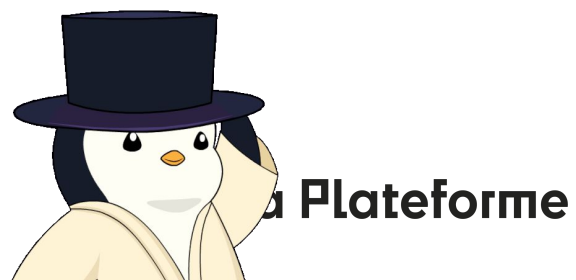
Tu fabriques d’abord un groupe de données dont tu sais qu’il respecte les règles. Ensuite, tu notes sa taille quelque part séparément. Puis tu mélanges ce groupe avec beaucoup de données aléatoires/inutiles pour le rendre difficile à repérer.

# Juge Arbitre

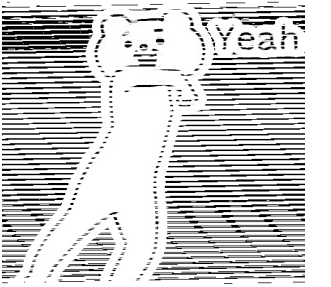
- Un arbitre en 5 lignes pour valider si un ensemble peut être livré sans aucun retard.
- **Théorème central** : un sous-ensemble est livrable à l'heure si et seulement si l'ordre par échéances croissantes ne cause aucun retard.
- Propriété fondamentale : la faisabilité ne dépend jamais du montant des pénalités.



- **Arbitre** : un petit algorithme qui regarde un groupe de tâches et répond « **oui, tout peut être terminé à temps** » ou « **non, il y aura du retard** ».
- **Théorème central** : pour savoir si c'est faisable, on trie les tâches par **date limite la plus proche** et on les exécute dans cet ordre. Si aucune n'est en retard → **c'est faisable**.
- **Pénalités** : le montant de la pénalité **ne change rien à la faisabilité**. Une tâche est soit terminée à temps, soit en retard, peu importe combien coûte son retard.



# Comparaison et performances



R1 (Pratique atelier)

88% de pénalités

R2 (Durée max)

52% de pénalités

R3 (Pénalité min)

36% de pénalités

R3' (Ratio w/p)

24% de pénalités

*Sur les carnets types, R3' réduit drastiquement les pertes financières par rapport à R1. Mais attention : sa moyenne cache des pires cas critiques !*

# Le Piège Adverse et Décision

## Démontrer les limites des heuristiques

À l'étape 5, vous devez construire une famille de carnets qui fait diverger R3 : une tâche longue bloque n tâches courtes prioritaires.

**Arbitrage d'ingénieur :** La meilleure règle en moyenne (R3') peut-elle être mise en échec ? Quelle décision défendrez-vous dans votre note d'une page face au chef d'atelier ?



# Modalités de Rendu et Rigueur

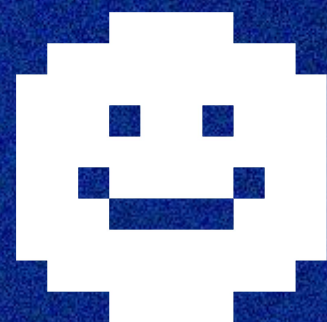
**Arborescence stricte** : Respectez l'architecture imposée (src/, carnets/, measures/, docs/, README.md) sur votre dépôt GitHub personnel.

**Règle bilingue** : Code source, identifiants et messages de commits en anglais. Rapport, démonstrations écrites et README en français.

**5 Oracles automatisés** : Votre programme doit vérifier seul les 5 propriétés fondamentales (ex.  $R2 \geq$  plantées, cohérence sur pénalités unitaires).

**Note de décision d'une page** : Une prise de position claire et argumentée sur la règle à déployer en production.





 La Plateforme